

L02- Deploying an AI Model on a Simulated Edge Device 2 Options

Martin Demel, Maira Tanweer Chachar, Jenner Ramos, Ryan Yaunch

Department of Science, Technology, Engineering & Math, Houston Community
College

ITAI-3377-A.I. at the Edge & IIOT Env-RT-17181

Professor Sitaram Ayyagari

January 28, 2026.

Introduction

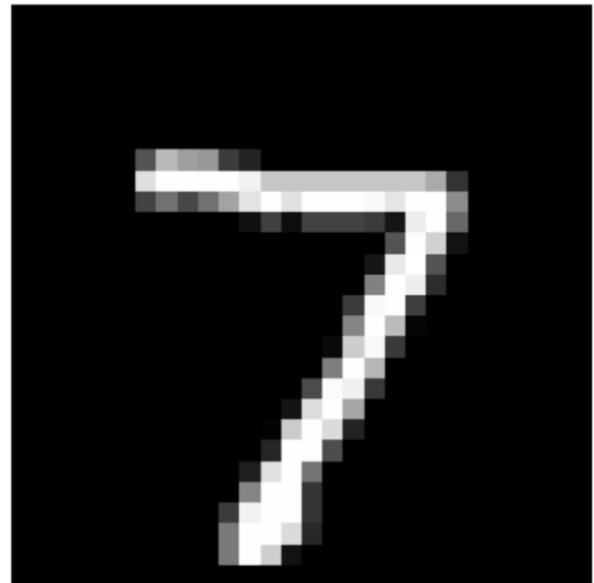
In this lab, our team set up a Python environment, trained a simple neural network on the MNIST dataset, converted the trained model to TensorFlow Lite format, and validated inference using the TensorFlow Lite Interpreter. The goal was to practice the end-to-end workflow for moving a trained model from development into a lightweight format suitable for deployment on devices with limited resources.

Body

From a learning perspective, we gained a deeper understanding of how preprocessing affects model inputs and why consistent shapes and data types are required. Normalizing MNIST images to values between zero and one improved training stability and enabled us to use a simple model architecture that still performed well. We also learned how the conversion process packages a trained model into an inference-optimized form and how the interpreter API exposes input and output details we can use to verify correct integration.

TensorFlow Lite is designed for real-world deployments where compute, memory, and power are constrained. In an Internet of Things setting, a small model can run locally on an edge device to classify sensor data or images without sending everything to the cloud. That approach reduces latency, improves responsiveness, and can strengthen privacy because raw data does not always

Predicted Label: 7, Actual Label: 7



need to leave the device. Our team also found that validation after conversion is essential, because a model that trains correctly is not automatically ready for deployment until inference is tested in the target runtime.

One of the biggest challenges we faced was keeping the development environment stable enough to complete the conversion step. During our attempts, we found that training the MNIST model was straightforward, but converting to TensorFlow Lite was sensitive to the specific TensorFlow and Keras versions installed. We addressed this by standardizing the environment and ensuring that TensorFlow and related packages were aligned so the converter could run without crashing. We also learned that capturing clear evidence of each step matters because conversion and inference can appear to work only after the output files are created, and the interpreter confirms the expected tensor shapes.

Working as a team helped us move faster through troubleshooting and documentation. We divided tasks among environment setup, model training, and reviewing conversion and inference outputs, and consolidated our findings into a single notebook workflow. This collaboration made it easier to spot issues early and confirm that each step produced the expected artifacts, such as the saved model file and the TensorFlow Lite file.

Conclusion

Overall, our team learned that deployment readiness is more than model accuracy. It includes repeatable environment setup, reliable conversion, and confirmed inference behavior in the runtime that will be used on-device. By training on MNIST and converting to TensorFlow Lite, we practiced the same conversion and validation process used in many edge and mobile AI applications.